NAVIS-AI: Vision-Driven Autonomous Navigation with A* Dynamic Rerouting

Gayathri Boopathy∗, Harish S∗†, Harshinee M∗†
∗Department of Computer Science, Rajalakshmi Engineering College, Chennai, Tamil Nadu, India
{gayathri.b, harish.s, harshinee.m}@rajalakshmi.edu.in
†Student IDs: 240701141 (Gayathri Boopathy), 240701174 (Harish S), 240701180 (Harshinee M)

***Abstract*—Conventional navigation depends on predefined road networks and the user's current GPS position, and it does not directly model temporary obstacles that appear in the environment. This paper presents NAVIS-AI, an autonomous navigation system that closes the loop between sensing, perception, and route planning. Live GPS supplies the user's position, a Firebase Realtime Database carries ESP32 destination telemetry, and a computer-vision perception layer detects obstacles and triggers route adaptation. When the active path is blocked, an A* search computes an alternate safe path. An operator dashboard shows the map, telemetry, perception status, event log, and an emergency-stop control. The prototype shows how live location, IoT telemetry, map routing, perception, and heuristic path planning can be combined in one interface, and it outlines a path toward YOLO-based detection and sensor-assisted heading estimation.**

***Index Terms*—autonomous navigation, A* search, obstacle detection, computer vision, dynamic rerouting, ESP32, Firebase, OpenStreetMap**

I. Introduction

Navigation systems are used to guide people and vehicles toward a destination, but most rely on a route that is computed once from a predefined road network and then followed using the current GPS position. When something changes along the way, such as a temporary obstacle, the route has to be corrected by hand or recomputed without any knowledge of what the environment now looks like. In localized and indoor settings the problem is greater, because additional sensing and a spatial representation of the surroundings are needed.

This work presents NAVIS-AI, an intelligent navigation platform that senses obstacles, updates the route, and guides the user toward a live destination. The main contributions are:

- An integrated control loop that links sensing, localization, obstacle detection, route blocking, A* search, rerouting, and navigation.
- A real-time data pipeline that combines live GPS with ESP32 destination telemetry exchanged through Firebase Realtime Database.
- An obstacle-aware A* planner that treats detected obstacles as blocked cells and returns an alternate safe path.
- An operator dashboard that presents the map, telemetry, perception status, event log, and an emergency-stop control on one screen.

The rest of the paper is organized as follows. Section II reviews related work, Section III describes the limitations of existing systems, Section IV introduces the proposed system, Section V details the architecture, Section VI explains the obstacle-aware A* method, Section VII describes the dashboard, and Sections VIII to XI cover the technology stack, limitations, future work, and conclusion.

II. Related Work

NAVIS-AI builds on established work in path planning, robotics, perception, and open mapping. The A* algorithm provides the graph-search foundation for optimal path planning [1]. Probabilistic robotics addresses localization, mapping, and uncertainty [2]. The YOLO family of detectors offers real-time object detection suitable for perception [3]. OpenStreetMap supplies open geographic data for visualization [4], and the Open Source Routing Machine (OSRM) provides road-network routing with distance and duration [5]. Firebase Realtime Database allows realtime exchange of ESP32 telemetry [6]. Table I summarizes these foundations.

TABLE I
Foundations Supporting NAVIS-AI

Area	Key Work / Technology	Relevance
A* search	Hart, Nilsson & Raphael (1968)	Graph-search foundation for optimal path planning
Probabilistic robotics	Thrun, Burgard & Fox	Localization, mapping, and uncertainty
YOLO	Redmon et al. (2016)	Real-time object detection for perception
OpenStreetMap	OSM ecosystem	Open geographic data for map visualization
OSRM	Open Source Routing Machine	Road-network routing, distance and duration
Firebase RTDB	Firebase Realtime Database	Realtime exchange of ESP32 telemetry

These components are mature individually, but they are often treated as separate parts of a navigation solution. An integrated pipeline that feeds perception results directly into route planning, and presents the outcome on a live map, is the gap that NAVIS-AI addresses.

III. Existing System and Limitations

Conventional navigation has several limitations that motivate this work:

- **Static route planning.** Traditional navigation primarily relies on predefined road networks and the current GPS position.
- **Limited obstacle modelling.** Static route planning does not directly model newly observed temporary obstacles.
- **Limited perception.** Indoor and localized navigation requires additional sensing and a spatial representation of the environment.

- **Fragmented data.** GPS, perception, and routing are often treated as separate components.
- **Slow adaptation.** Manual rerouting can introduce delay when the environment changes.

IV. Proposed System

NAVIS-AI closes the loop between sensing, routing, and live navigation. It rests on four elements: live GPS that drives a moving user marker, an ESP32 target whose destination is published through Firebase, vision-based obstacle perception, and A* search that produces an alternate safe path.

The system operates as a continuous control loop with the following stages:

1) **Sense:** read live GPS and ESP32 telemetry, and capture the camera feed.
2) **Localize:** place the user and the destination on the map.
3) **Detect:** identify obstacles in the camera view.
4) **Block:** convert a detected obstacle into blocked cells that invalidate the active path.
5) **A*:** search for a safe alternative around the blocked cells.
6) **Reroute:** replace the active route with the new path.
7) **Navigate:** guide the user along the updated route on the live map.

V. System Architecture

Fig. 1 shows the end-to-end data flow. GPS, ESP32 telemetry, and camera input are combined into obstacle constraints, which feed the A* search and finally the live map.

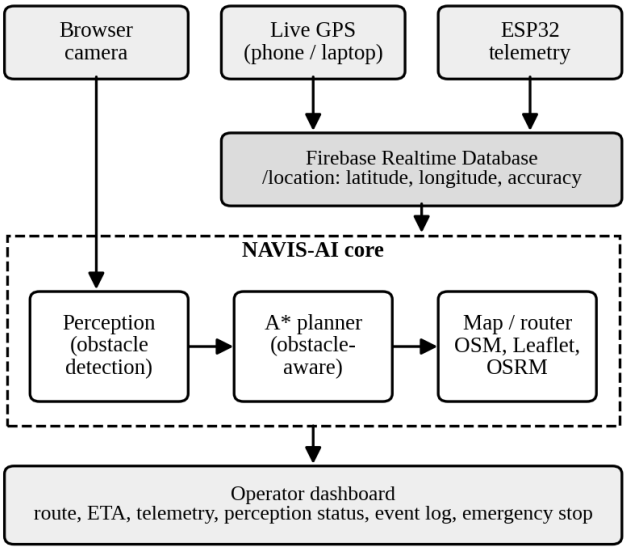


Fig. 1. End-to-end data flow of NAVIS-AI.

A. Positioning and Telemetry

A laptop or phone supplies the live GPS position. Location records consisting of latitude, longitude, and accuracy are exchanged through the `/location` node of the Firebase Realtime Database, which also carries the ESP32 destination telemetry to the web dashboard [6].

B. Perception

The browser camera provides the visual input. The perception layer detects obstacles and reports them to the planner as perception events, which are also written to the event log.

C. Planning and Mapping

The A* planner computes an alternate safe path when the active path is blocked. Map rendering uses OpenStreetMap tiles displayed with Leaflet [4][7], while OSRM supplies the road route together with its distance and duration [5].

D. Operator Dashboard

The dashboard gathers the map, telemetry, perception status, event log, and controls into a single interface, described further in Section VII.

VI. Methodology: Obstacle-Aware A*

When an obstacle blocks the active path, NAVIS-AI searches for a safer alternative using A* [1]. The search expands nodes in order of the priority score

$$f(n) = g(n) + h(n) \quad (1)$$

where $g(n)$ is the cost from the start, $h(n)$ is the estimated cost to the goal, and $f(n)$ is the total priority score. With an admissible heuristic, A* returns a minimum-cost path [1].

A. Grid and Obstacle Model

The area between the user and the destination is represented as a grid. Cells that coincide with a detected obstacle are marked as blocked and are excluded from the search. The current route, the obstacle, the blocked cells, the A* search, and the resulting safe route form the sequence illustrated in Fig. 2.

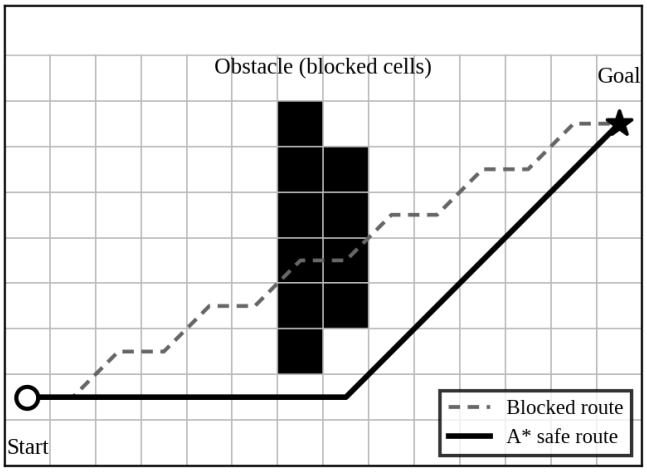


Fig. 2. Illustrative grid example of obstacle-aware rerouting.

B. Rerouting Procedure

1) Compute the current route from the user's live position to the destination.
2) When the perception layer reports an obstacle that intersects the route, mark the affected cells as blocked.
3) Run A* from the current position to the destination over the unblocked cells.
4) Display the alternate safe path on the map and record the event in the event log.

VII. Operator Dashboard

The dashboard is designed for operators who need live state and quick decisions. Its design principle is one screen, live state, fast decisions, and explainable events. Table II lists its features.

TABLE II
NAVIS-AI Dashboard Features

Feature	Description
Live GPS	Moving user marker
ESP32 target	Destination received from Firebase
Road route	Distance and ETA
Obstacles	Perception events
A* reroute	Alternate path around blocked cells
Event log	System trace of events
Head direction	IMU-ready heading display
Emergency stop	Navigation halt

VIII. Technology Stack

The prototype combines the following technologies:

- **ESP32** for destination telemetry.
- **Firebase Realtime Database** for realtime communication with the dashboard.
- **Live GPS** from a laptop or phone for user positioning.
- **Leaflet and OpenStreetMap** for map visualization.
- **OSRM** for road routing, distance, and duration.
- **Computer vision** for obstacle perception.
- **A* path planning** for alternate-route computation.

IX. Current Status and Limitations

This paper describes the design and prototype behaviour of NAVIS-AI. The prototype demonstrates that an obstacle can invalidate the active path and trigger an alternate route, and that Firebase provides realtime communication between ESP32 telemetry and the web dashboard. A quantitative evaluation, such as detection accuracy, rerouting latency, and path-length overhead, has not yet been carried out. In addition, independent ESP32 positioning and heading estimation depend on hardware that is planned but not yet integrated, and the link between detected objects and road-segment constraints remains to be developed.

X. Future Work

- **Heading estimation:** add an MPU9250 sensor to provide heading, building on the IMU-ready head-direction display.
- **Independent positioning:** add a GPS module so that the ESP32 can report its own position.
- **Smarter perception:** use OpenCV or YOLO detection linked to road-segment constraints for smarter A* planning.

XI. Conclusion

NAVIS-AI combines live location, IoT telemetry, map routing, perception, and A* planning into one navigation platform. An obstacle detected by the camera can invalidate the active path and trigger an alternate route, while Firebase keeps ESP32 telemetry and the web dashboard in step. The work moves a navigation prototype toward perception-aware autonomous routing, and the planned hardware and detection upgrades will make the planner more capable and the system less dependent on external positioning.

Acknowledgment

The authors thank their faculty mentors and peers for feedback during the design and development of this project.

References

[1] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," IEEE Trans. Syst. Sci. Cybern., vol. 4, no. 2, pp. 100–107, 1968.

[2] S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics. Cambridge, MA, USA: MIT Press, 2005.

[3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 779–788.

[4] OpenStreetMap contributors, "OpenStreetMap." [Online]. Available: https://www.openstreetmap.org

[5] D. Luxen and C. Vetter, "Real-time routing with OpenStreetMap data," in Proc. 19th ACM SIGSPATIAL Int. Conf. Adv. Geogr. Inf. Syst., 2011, pp. 513–516.

[6] Google, "Firebase Realtime Database documentation." [Online]. Available: https://firebase.google.com/docs/database

[7] V. Agafonkin, "Leaflet: An open-source JavaScript library for interactive maps." [Online]. Available: https://leafletjs.com